```
MODULE IntegrationContract
EXTENDS Naturals, FiniteSets, FailurePolicy

CONSTANTS Hosts, Checks, Names, MaxRequests, MaxChanges, MaxSessions

AllHosts ≜ {"claude", "codex", "opencode1", "opencode2", "pi", "neovim"}
ASSUME Hosts ⊆ AllHosts ∧ Hosts ≠ {}
```

These are host capabilities, not advisory/strict policy modes. A warning cannot be counted as a denied completion, and an editor has no agent gate.

```
Capabilities(h) ≜
    [agent ↦ h ≠ "neovim",
     waitsForCheck ↦ h ∈ {"codex", "opencode1", "opencode2", "pi"},
     completion ↦ CASE h ∈ {"claude", "codex"} → "block"
                    □ h = "opencode1" → "warn"
                    □ h = "opencode2" → "queue"
                    □ h = "pi" → "continue"
                    □ OTHER → "none",
     delivery ↦ CASE h = "claude" → "turn"
                  □ h = "opencode2" → "context"
                  □ h = "neovim" → "quickfix"
                  □ OTHER → "event",
     idleWake ↦ h ∈ {"claude", "opencode2"},
     status ↦ h ∈ {"claude", "pi", "neovim"}]

Kinds ≜ {"inspect", "repair", "publish", "complete"}
Empty ≜ [c ∈ Checks ↦ {}]
Snapshots ≜ [Checks → SUBSET Names]
NoReply ≜ [ready ↦ FALSE, decision ↦ "none"]

VARIABLES host, active, busy, reports, baseline, inherited, selected,
          request, reply, requests, changes, session,
          visible, contextPending, contextSent, correctionQueued,
          contexts, warnings, corrections, inTurn, wakes, work

vars ≜ ⟨host, active, busy, reports, baseline, inherited, selected,
        request, reply, requests, changes, session, visible,
        contextPending, contextSent, correctionQueued, contexts,
        warnings, corrections, inTurn, wakes, work⟩

Guarded(scope) ≜
    {c ∈ scope : FailureGuards(reports[c], baseline[c], inherited)}
Guard ≜ Capabilities(host).agent ∧ Guarded(Checks) ≠ {}
Notify ≜ Capabilities(host).agent ∧
         (∃ c ∈ Checks : FailureGuards(reports[c], baseline[c], "allow"))
SelectedChecks ≜ IF selected = {} THEN Checks ELSE selected

TypeOK ≜
    ∧ host ∈ Hosts ∧ active ∈ BOOLEAN ∧ busy ∈ BOOLEAN
    ∧ work ∈ 0 .. ((MaxSessions + 1) * MaxRequests)
    ∧ inTurn ∈ BOOLEAN
    ∧ wakes ∈ 0 .. (MaxChanges + MaxSessions + 1 + (MaxSessions + 1) * MaxRequests)
    ∧ reports ∈ Snapshots ∧ baseline ∈ Snapshots
    ∧ inherited ∈ {"allow", "block"} ∧ selected ⊆ Checks
    ∧ request ∈ Kinds ∪ {"none"}
    ∧ reply ∈ [ready : BOOLEAN ,
               decision : {"none", "allow", "deny", "block", "warn", "queue", "continue", "cancelled"}]
    ∧ requests ∈ 0 .. MaxRequests ∧ changes ∈ 0 .. MaxChanges ∧ session ∈ 0 .. MaxSessions
    ∧ visible ∈ [known : BOOLEAN , failures : Snapshots]
```

```
    ∧   contextPending ∈ BOOLEAN  ∧ contextSent ∈ BOOLEAN  ∧ correctionQueued ∈ BOOLEAN
    ∧   contexts  ∈ 0 .. (MaxChanges + MaxSessions + 1)
    ∧   warnings ∈ 0 .. ((MaxSessions + 1) * MaxRequests)
    ∧   corrections ∈ 0 .. ((MaxSessions + 1) * MaxRequests)

Init ≜
    ∧ host ∈ Hosts ∧ active = TRUE ∧ busy ∈ BOOLEAN
    ∧ reports ∈ Snapshots ∧ baseline ∈ Snapshots
    ∧ inherited ∈ {"allow", "block"} ∧ selected ∈ SUBSET Checks
    ∧ request = "none" ∧ reply = NoReply
    ∧ requests = 0 ∧ changes = 0 ∧ session = 0
    ∧ visible = [known ↦ FALSE, failures ↦ Empty]
    ∧ contextPending = FALSE ∧ contextSent = FALSE ∧ correctionQueued = FALSE
    ∧ contexts = 0 ∧ warnings = 0 ∧ corrections = 0 ∧ wakes = 0 ∧ work = 0 ∧ inTurn ∈ BOOLEAN

Request(k) ≜
    ∧   active ∧ request = "none" ∧ requests < MaxRequests
    ∧   request′ = k ∧ reply′ = NoReply ∧ requests′ = requests + 1
    ∧   UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                   changes, session, visible, contextPending, contextSent,
                   correctionQueued, contexts, warnings, corrections, inTurn, wakes, work⟩

CanRespond ≜
    active ∧ request ≠ "none" ∧ ¬reply.ready ∧
    (request ∈ {"inspect", "repair"}      ∨ ¬Capabilities(host).agent ∨
     ¬Capabilities(host).waitsForCheck ∨ ¬busy)

Decision ≜
    CASE request ∈ {"inspect", "repair"} → "allow"
      □   request = "publish" →
            IF Capabilities(host).agent ∧ Guarded(SelectedChecks) ≠ {}
             THEN "deny" ELSE "allow"
      □   request = "complete" → IF Guard THEN Capabilities(host).completion ELSE "allow"

QueuesCorrection ≜ Decision = "queue" ∨ (host = "claude" ∧ Decision = "block")

Respond ≜
    ∧ CanRespond
    ∧ inTurn′ = IF request = "complete" ∧
                   (Decision ∈ {"allow", "warn", "queue"} ∨ host = "claude")
                THEN FALSE ELSE inTurn
    ∧ reply′ = [ready ↦ TRUE, decision ↦ Decision]
    ∧ warnings′ = warnings + IF Decision = "warn" THEN 1 ELSE 0
    ∧ corrections′ = corrections + IF QueuesCorrection ∧ ¬correctionQueued THEN 1 ELSE 0
    ∧ correctionQueued′ = (correctionQueued ∨ QueuesCorrection)
    ∧ wakes′ = wakes + IF QueuesCorrection ∧ ¬correctionQueued ∧
                          (host = "opencode2" ∨ ¬inTurn)
                       THEN 1 ELSE 0
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 request, requests, changes, session, visible,
                 contextPending, contextSent, contexts, work⟩

Acknowledge ≜
    ∧ request ≠ "none" ∧ reply.ready
    ∧ request′ = "none"
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 reply, requests, changes, session, visible,
                 contextPending, contextSent, correctionQueued,
                 contexts, warnings, corrections, inTurn, wakes, work⟩
```

```
BeginCheck ≜
    ∧ active ∧ ¬busy ∧ changes < MaxChanges
    ∧ busy′ = TRUE
    ∧ UNCHANGED ⟨host, active, reports, baseline, inherited, selected,
                 request, reply, requests, changes, session, visible,
                 contextPending, contextSent, correctionQueued,
                 contexts, warnings, corrections, inTurn, wakes, work⟩
```

The result-ordering and baseline rules are specified in *Overwatch.tla*. This layer accepts a completed result at that boundary, then constrains what every host must expose and which requests it may hold behind a check.

```
FinishCheck(f) ≜
    ∧ active ∧ busy ∧ changes < MaxChanges
    ∧ reports′ = f ∧ busy′ = FALSE ∧ changes′ = changes + 1
    ∧ IF f ≠ reports
        THEN ∧ contextSent′ = FALSE ∧ correctionQueued′ = FALSE
             ∧ contextPending′ = FALSE
        ELSE UNCHANGED ⟨contextSent, correctionQueued, contextPending⟩
    ∧ UNCHANGED ⟨host, active, baseline, inherited, selected, request,
                 reply, requests, session, visible, contexts, warnings, corrections, inTurn, wakes, work⟩

Report ≜
    ∧ active ∧ ¬busy
    ∧ visible′ = [known ↦ TRUE, failures ↦ reports]
    ∧ contextPending′ = (Notify ∧ ¬contextSent ∧ Capabilities(host).delivery ∈ {"turn", "context"})
    ∧ contextSent′ = (contextSent ∨ (Notify ∧ Capabilities(host).delivery = "event"))
    ∧ contexts′ = contexts + IF Notify ∧ ¬contextSent ∧ Capabilities(host).delivery = "event" THEN 1 ELSE 0
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 request, reply, requests, changes, session,
                 correctionQueued, warnings, corrections, inTurn, wakes, work⟩

Deliver ≜
    ∧ active ∧ contextPending
    ∧ wakes′ = wakes + IF host = "claude" ∧ ¬inTurn THEN 1 ELSE 0
    ∧ contextSent′ = TRUE ∧ contextPending′ = FALSE ∧ contexts′ = contexts + 1
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 request, reply, requests, changes, session, visible,
                 correctionQueued, warnings, corrections, inTurn, work⟩

Stop ≜
    ∧ active
    ∧ inTurn′ = FALSE
    ∧ active′ = FALSE ∧ busy′ = FALSE ∧ request′ = "none"
    ∧ reply′ = [ready ↦ TRUE, decision ↦ "cancelled"]
    ∧ visible′ = [known ↦ FALSE, failures ↦ Empty]
    ∧ contextPending′ = FALSE
    ∧ UNCHANGED ⟨host, reports, baseline, inherited, selected, requests,
                 changes, session, contextSent, correctionQueued,
                 contexts, warnings, corrections, wakes, work⟩

Restart ≜
    ∧ ¬active ∧ session < MaxSessions
    ∧ inTurn′ = FALSE
    ∧ active′ = TRUE ∧ session′ = session + 1 ∧ busy′ = FALSE
    ∧ baseline′ = reports
    ∧ request′ = "none" ∧ reply′ = NoReply ∧ requests′ = 0
    ∧ visible′ = [known ↦ FALSE, failures ↦ Empty]
    ∧ contextPending′ = FALSE ∧ contextSent′ = FALSE ∧ correctionQueued′ = FALSE
    ∧ UNCHANGED ⟨host, reports, inherited, selected, changes, contexts, warnings, corrections, wakes, work⟩
```

$StartTurn \triangleq$
  $\land\ active \land \neg inTurn \land inTurn' = \text{TRUE}$
  $\land\ \text{UNCHANGED}\ \langle host, active, busy, reports, baseline, inherited, selected,$
  $request, reply, requests, changes, session, visible,$
  $contextPending, contextSent, correctionQueued, contexts,$
  $warnings, corrections, wakes, work\rangle$

$WorkCompleted \triangleq$
  $\land\ active \land work < (MaxSessions + 1) * MaxRequests$
  $\land\ work' = work + 1 \land correctionQueued' = \text{FALSE}$
  $\land\ \text{UNCHANGED}\ \langle host, active, busy, reports, baseline, inherited, selected,$
  $request, reply, requests, changes, session, visible,$
  $contextPending, contextSent, contexts, warnings, corrections, inTurn, wakes\rangle$

$CheckCompletes \triangleq \exists f \in Snapshots : FinishCheck(f)$
$Next \triangleq (\exists k \in Kinds : Request(k)) \lor Respond \lor Acknowledge \lor BeginCheck$
  $\lor CheckCompletes \lor Report \lor Deliver \lor Stop \lor Restart \lor StartTurn \lor WorkCompleted$
  $\lor (\neg active \land session = MaxSessions \land \text{UNCHANGED}\ vars)$

Check completion fairness is the *RunsEnd* guarantee of the result model. Inspection is independently enabled; it never needs that guarantee to reply.

$Spec \triangleq Init \land \Box[Next]_{vars} \land \text{WF}_{vars}(CheckCompletes)$
  $\land \text{WF}_{vars}(Respond) \land \text{WF}_{vars}(Report) \land \text{WF}_{vars}(Deliver)$

$InspectionIndependentOfChecks \triangleq$
  $active \land request \in \{\text{“inspect”}, \text{“repair”}\} \land \neg reply.ready \Rightarrow \text{ENABLED}\ Respond$
$InspectionAlwaysAllowed \triangleq$
  $request \in \{\text{“inspect”}, \text{“repair”}\} \land reply.ready \Rightarrow reply.decision = \text{“allow”}$
$EditorNeverGuards \triangleq$
  $host = \text{“neovim”} \land request \neq \text{“none”} \land reply.ready \Rightarrow reply.decision = \text{“allow”}$
$StoppedSessionsStaySilent \triangleq \neg active \Rightarrow \neg contextPending \land \neg visible.known$
$HostsWithoutIdleWakeStayIdle \triangleq \neg Capabilities(host).idleWake \Rightarrow wakes = 0$
$CorrectionRequestsAreDeduplicated \triangleq corrections \leq changes + session + work + 1$
$PendingContextIsCurrent \triangleq contextPending \Rightarrow Notify$
$CompletionUsesHostCapability \triangleq$
  $request = \text{“complete”} \land reply.ready \Rightarrow$
    $reply.decision \in \{\text{“allow”}, Capabilities(host).completion\}$
$EditorOnlyReports \triangleq$
  $host = \text{“neovim”} \Rightarrow contexts = 0 \land warnings = 0 \land corrections = 0 \land wakes = 0$
$PendingReportsEventuallyResolve \triangleq contextPending \leadsto \neg contextPending$
$FinishedReportsBecomeVisible \triangleq$
  $(active \land \neg busy) \leadsto (visible.known \lor \neg active)$
$InspectionEventuallyReplies \triangleq$
  $(active \land request \in \{\text{“inspect”}, \text{“repair”}\} \land \neg reply.ready)$
    $\leadsto (reply.ready \lor \neg active)$